

ScaleRAG

Multimodal, Hierarchical RAG for Scientific Papers

[GitHub Repo Link](#)

Team

Mahdi Tabesh • Manush Kalwari

Columbia University • EE Department • Final Project



Agenda

- Motivation and Problem Definition
- Technical Approach
- Results & Evaluation
- Demo or Visualization
- Lessons Learned
- Next Steps

Motivation & Overview

Pain points in scientific PDFs

- Long documents with cross-section dependencies
→ **flat chunking breaks global context**
- Figures and tables often contain the *answer*
→ **text-only RAG misses critical evidence**
- At arXiv scale, flat retrieval degrades
→ **recall–latency–memory trade-offs become prohibitive**

Our goal

- **Grounded QA over 1K+ papers with predictable latency at scale**
- **Preserve document structure** (paper → section → chunk)
- **Selectively incorporate visual evidence** under realistic compute constraints

Hierarchical coarse-to-fine retrieval, with optional multimodal evidence at the finest level

Corpus & preprocessing

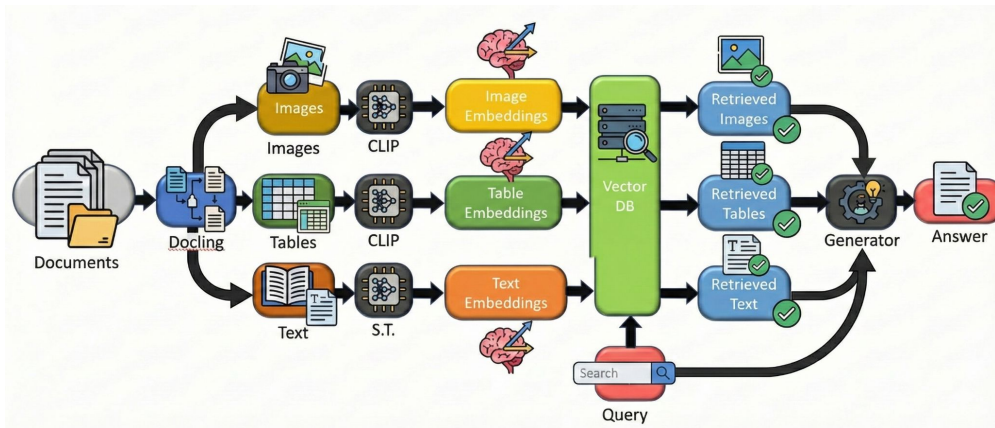
- **1,023 recent arXiv ML-systems papers** (PDFs)
- **Docling-based layout parsing**
 - headings, paragraphs, figures, tables, equations
- **Layout-aware chunk merging**
 - page + section boundaries, token-bounded
- **Paragraph-level chunking**
 - preserves local coherence
- **Figures/tables retained as chunks**
 - prevents evidence loss
- **Author blocks, affiliations, references removed**
 - reduces retrieval noise

Evaluation

- **Retrieval quality:** Precision@5, Recall@5, nDCG@5
- **Latency:** retrieval + generation (**single-GPU**)
- **End-to-end grounding accuracy**
 - answers supported by retrieved evidence
- **Synthetic QA dataset (self-instruct)**
 - controlled, slice-grounded questions

We compare 3 configs: (1) flat text RAG (baseline), (2) multimodal RAG (text+figures), (3) hierarchical ScaleRAG

APPROACH -I & II

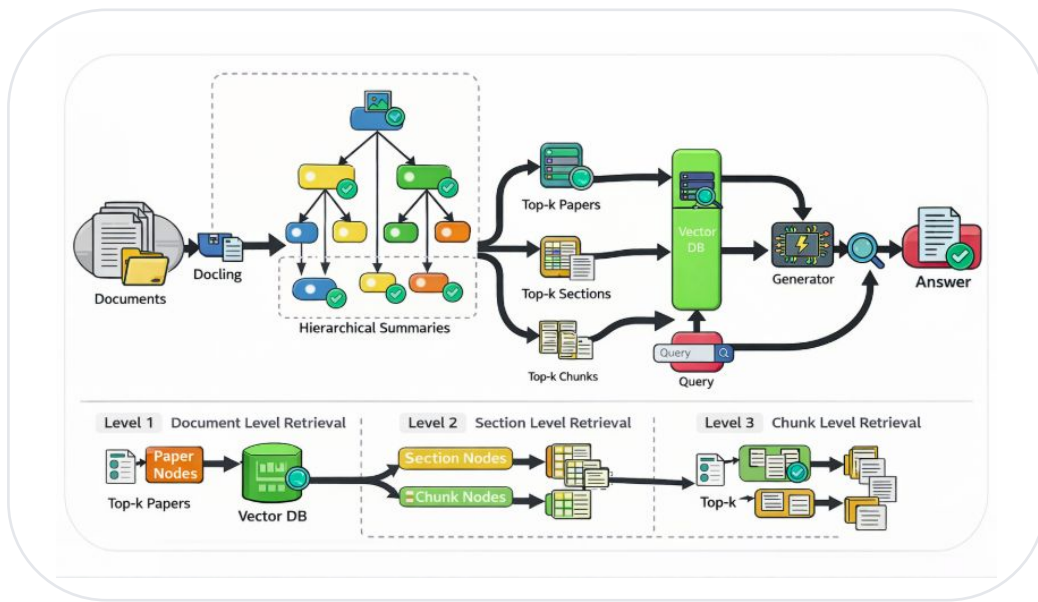


Intuition

Retrieve complementary evidence from text and visuals, enabling answers that require figures or tables that text-only RAG would miss.

What's special here

- **Modality-aware document parsing** (text, figures, tables via Docling)
- **Multimodal indexing**
→ text embeddings + CLIP vision embeddings (figures & tables)
- **Flat retrieval across modalities**
→ top-k text and visual chunks per query
- **Separate FAISS indexes per modality**
→ cosine similarity search
- **Multimodal answer generation**
→ LLaVA-1.5-7B (4-bit) conditioned on retrieved text + images

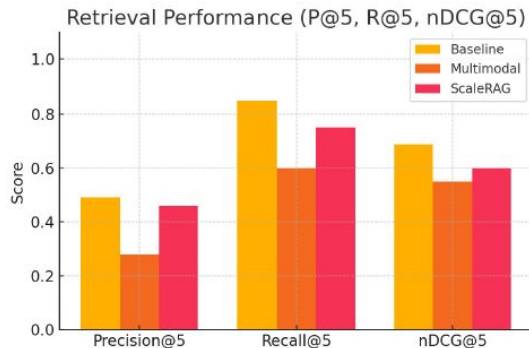


Intuition

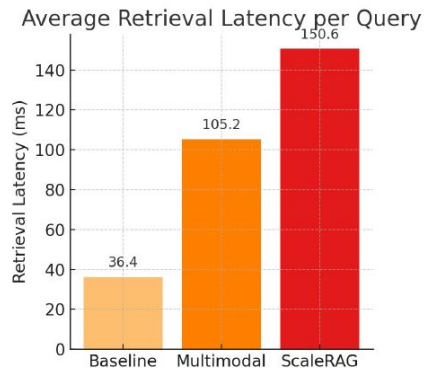
First retrieve globally relevant papers and sections using summaries, then drill down to precise evidence at the chunk level.

What's special here

- **Hierarchical document representation**
→ L0 raw chunks → L1 section summaries → L2 paper-level summaries
- **Offline summarization with LLMs**
→ GPT-4o-mini used to generate L1 and L2 nodes per paper
- **Multi-level vector indexing**
→ Separate FAISS indexes for L1 and L2 summary embeddings
- **Coarse-to-fine retrieval**
→ retrieve top-k papers → sections → fine-grained chunks
- **Efficient scaling to 1K+ papers**
→ avoids querying all chunks at inference time



Retrieval (avg, k=5)

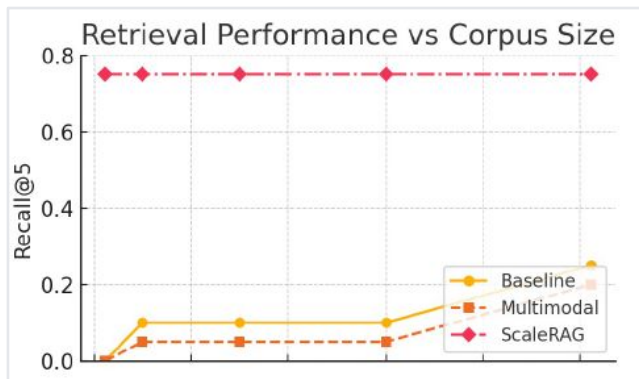


Retrieval latency per query

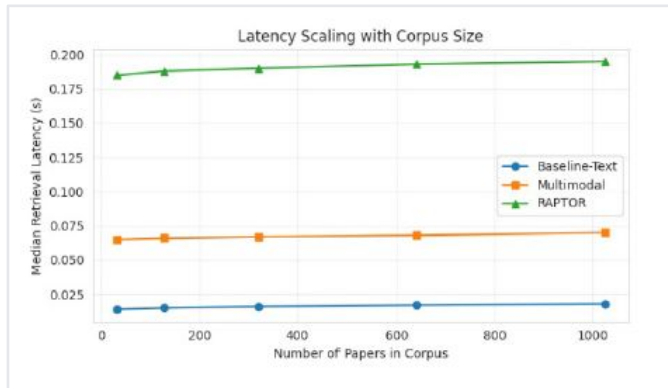
Takeaways

- Flat text RAG is strongest for **text-centric queries**
- Multimodal retrieval adds cost and noise without consistent gains
- Hierarchical retrieval incurs higher latency but remains competitive in quality

Trade-off: higher constant overhead → more stable retrieval at scale



Retrieval effectiveness vs corpus size (Fig. 7)



Median retrieval latency vs corpus size (Fig. 6)

Takeaways

Flat & multimodal RAG do not scale

- Recall remains low and unreliable, despite slight increases with corpus size

Hierarchical (ScaleRAG) retrieval is robust

- Maintains ~75% Recall@5 and stable nDCG

Latency remains predictable

- Higher constant cost, but no growth with corpus size

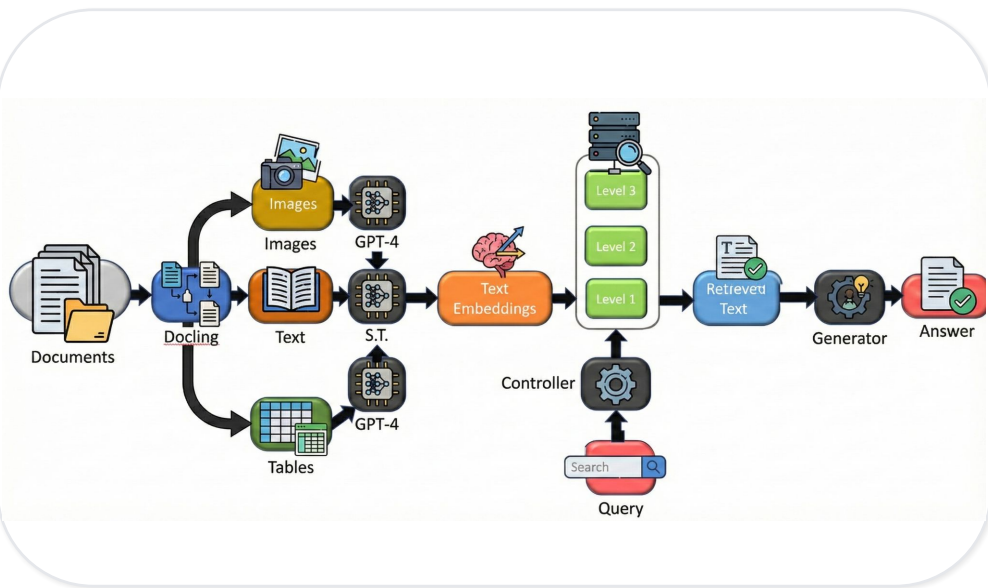
Key trade-off: *Small-scale speed vs. large-scale reliability*

Query: How does vLLM improve throughput for LLM inference?

Baseline (Flat RAG)	“The vLLM engine introduces a novel PagedAttention mechanism to better utilize GPU memory, which significantly reduces idle time. This optimization allows vLLM to achieve 2× higher throughput compared to a standard transformer serving pipeline 23.”
Multimodal (Text + Image)	“The vLLM system uses an improved attention and memory management scheme to speed up inference. By optimizing how data is stored and accessed, it achieves better throughput during LLM serving.”
ScaleRAG (Raptor)	“vLLM improves inference throughput by using PagedAttention to manage GPU memory more efficiently. This technique lets the model fetch and interleave data without costly memory copies, dramatically increasing throughput (on the order of double) over conventional approaches 23.”

- Baseline & ScaleRAG retrieve the correct paper and cite the key claim (*~2× throughput via PagedAttention*)
- Multimodal RAG fails silently: produces a plausible answer without grounding or citation
- Hierarchy improves evidence recall, not just answer fluency

APPROACH -III



Intuition

Hierarchical RAG trades a single expensive global search for multiple cheap, targeted searches.

← PIPELINE →

- **Document Parsing**
Parse PDFs into text, figures, and tables.
Convert non-text into textual summaries.
- **Hierarchical Representation**
Organize content at three granularities:
Document (L1) → Page (L2) → Chunk (L3)
- **Multi-Level Embeddings**
Build FAISS indices offline.
- **Adaptive Controller**
Based on query complexity and intent.
- **Selective Retrieval**
Traverse relevant indices avoiding global scans.
- **Generation**
Feed retrieved context to the generator to produce the final answer.

3-level retrieval stack

L1: Paper embeddings

Fast shortlist of candidate papers



L2: Page embeddings

Narrow to relevant pages



L3: Chunk embeddings

Precise evidence for final context

Adaptive depth: stop early when L1/L2 is sufficient

Staged evaluation plan

- Step 1: Baseline flat RAG on 20 papers
- Step 2: Add hierarchy (L1/L2/L3) on same 20 papers
- Step 3: Scale to 1023 papers with identical pipeline
- Judge answer quality automatically (faithfulness + usefulness)
- QnA Pipeline



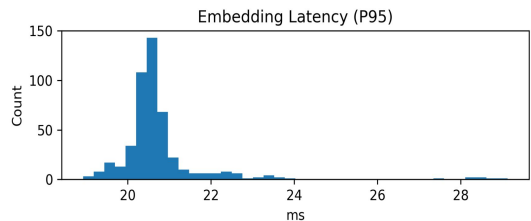
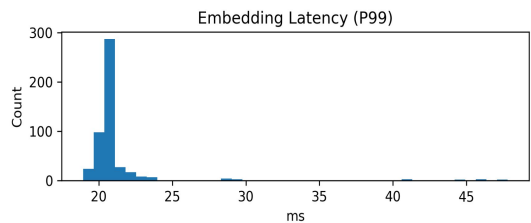
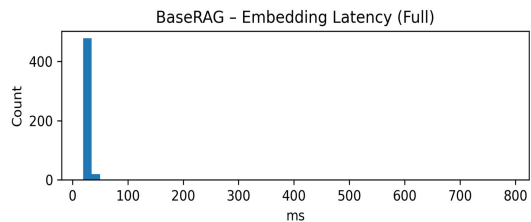
```
SELF_INSTRUCT_PROMPT = """
You are a helpful assistant reading a research paper excerpt.

TEXT:
\"\"\"{text}\"\"\"

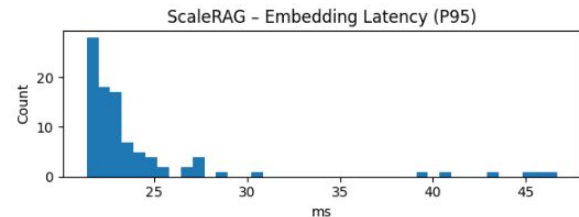
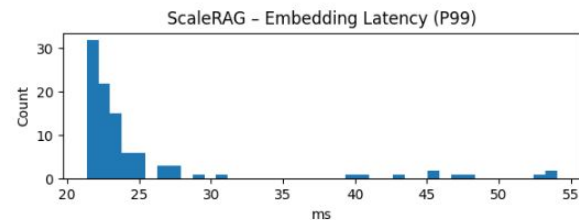
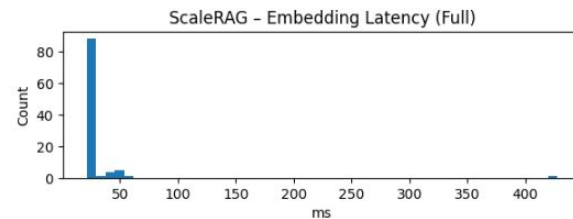
Generate {n_qas} diverse question-answer pairs that can be answered *directly and unambiguously* from this text alone.

Requirements:
- Cover different types: definitions, methods, motivations, comparisons, results.
- Make questions specific, not vague.
- Answers should be concise and copy or paraphrase the text.
- Return JSON as a list under key "qas", like:
  {{ "qas": [{{ "question": "...", "answer": "..." }}, ...] }}
"""
```

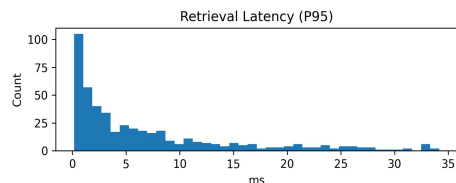
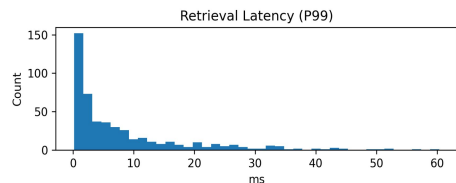
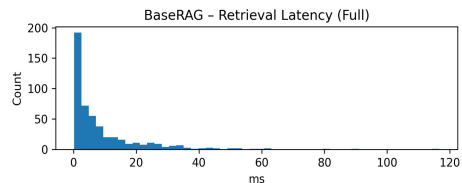
Base-RAG



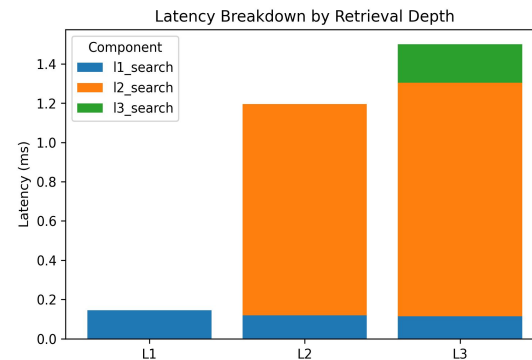
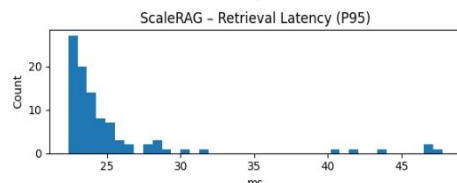
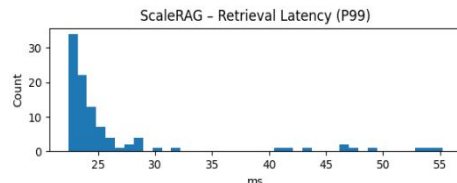
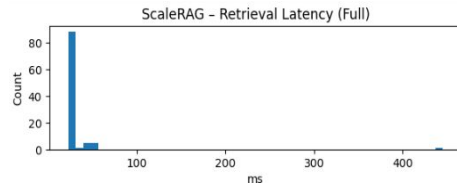
Scale-RAG



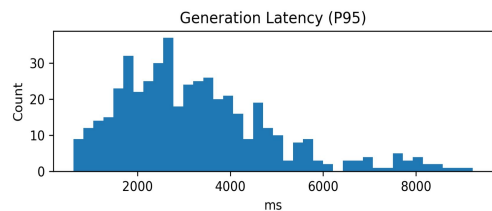
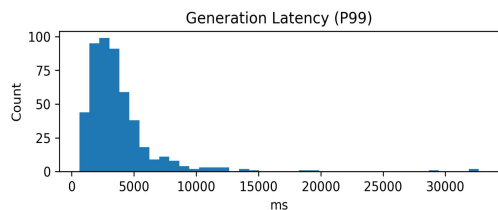
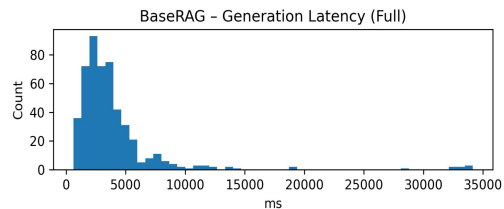
Base-RAG



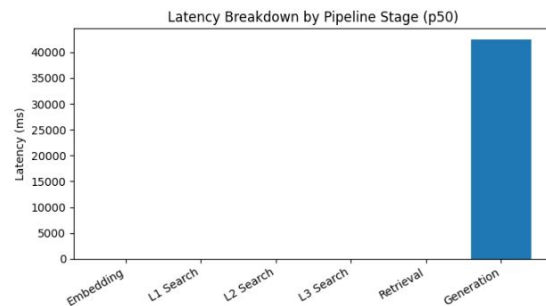
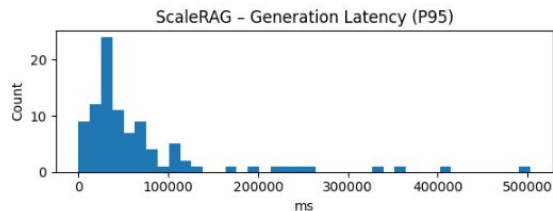
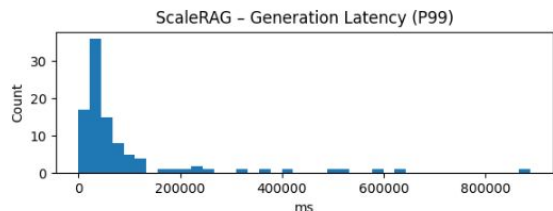
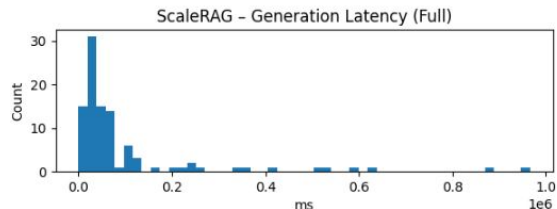
Scale-RAG



Base-RAG



Scale-RAG



METRIC	BaseRAG	ScaleRAG
Hit@5	0.800	1.000
Recall@5	0.800	1.000
DCG@5	2.7735	2.948
Answer Score	0.625	0.840

Scored by GPT-5-nano

```
prompt = f"""
    You are evaluating a Retrieval-Augmented Generation (RAG) system.

    QUESTION:
    {query}

    GOLD ANSWER:
    {gold_answer}

    MODEL ANSWER:
    {pred_answer}

    RETRIEVED CHUNKS (ranked best to worst):
    {chunk_text}

    TASKS:
    1. For each chunk, assign a relevance score:
       - 2 = directly supports the gold answer
       - 1 = partially relevant
       - 0 = irrelevant

    2. Score the MODEL ANSWER from 0 to 1 based on correctness vs GOLD ANSWER.

    Return JSON ONLY in this format:
    {{
      "chunk_relevance": [r1, r2, r3, r4, r5],
      "answer_score": float
    }}
    """
```

Lessons Learned

Lessons learned →

- **Hierarchy trades search cost for control.**
Reduces search space, but additional levels introduce latency overhead.
- **Generation dominates latency (~99%).**
Retrieval optimizations help, but end-to-end speed is bottlenecked by decoding. Generation-side optimizations matter most.
- **Most questions don't need full depth.**
Coarse, section-level context answers the majority of queries



	retrieval_level
1	3.0%
2	69.0%
3	28.0%

Next steps →

- **Targeting latency scaling.**
Integrate speculative decoding or Medusa heads to reduce generation latency.
- **Learned Routing instead of heuristics.**
Train a lightweight policy to decide retrieval depth based on query.
- **Scale beyond 1K papers.**
Evaluate approximate and distributed indexing at larger corpus sizes.

Who did what

Student	Contribution
Mahdi Tabesh	Multimodal RAG, RAPTOR
Manush Kalwari	QnA Harness, Hierarchical RAG

How we collaborated

- Common corpus and Parsing pipeline so results are comparable
- Shared QnA dataset and ran ablations on same Evaluation harness. Used standardized metrics
- Collaborated on Final Report and Presentation Slides

Thank You